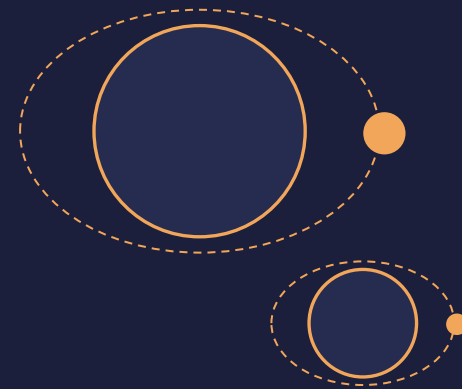


COMPUTAÇÃO GRÁFICA

Sistemas de Coordenadas

De Ptolomeu ao pipeline gráfico: por que o referencial que você escolhe muda tudo



Sumário

Sistemas de Coordenadas em Computação Gráfica

1

Motivação

Da pergunta disparadora ao paralelo com Copérnico

2

Geocêntrico × heliocêntrico

Ptolomeu, Aristóteles, Copérnico, Kepler e Galileu

3

Tipos de coordenadas

Cartesianas, polares, cilíndricas e esféricas

4

Fundamentos matemáticos

Espaços 2D/3D, orientação de eixos, mão direita × esquerda

5

Espaços do pipeline gráfico

Objeto, mundo, câmera, projeção e tela

6

Transformações na prática

Rotação e translação num exemplo guiado

7

Fechamento

Síntese e próximos passos

Objetivos de aprendizagem

Ao final desta aula, você será capaz de:

- 1 Entender por que a escolha de um sistema de coordenadas muda a complexidade de uma descrição matemática
- 2 Diferenciar coordenadas cartesianas, polares, cilíndricas e esféricas — e saber quando cada uma simplifica um problema
- 3 Diferenciar os sistemas de coordenadas do pipeline gráfico: objeto, mundo, câmera, projeção e tela
- 4 Rastrear manualmente um ponto do espaço do objeto até o espaço de tela



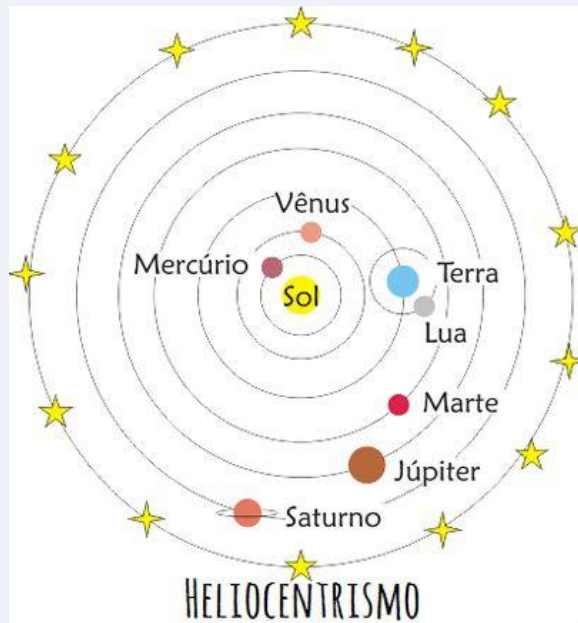
“Se eu desenho um cubo no Blender e depois o vejo na tela do meu monitor, quantos sistemas de referência diferentes estão envolvidos até aquele pixel acender?”

Mais do que parece — e é exatamente isso que vamos desmontar hoje.

Um mesmo céu, dois referenciais

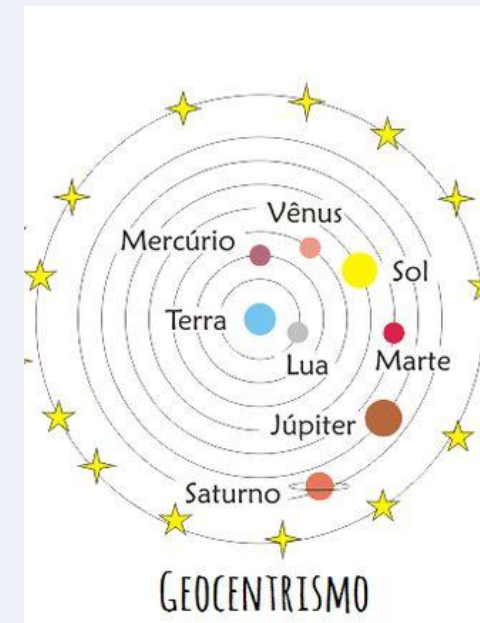
Por que os antigos achavam que a Terra era o centro do universo?

Geocêntrico (Ptolomeu)



+ epíclis para explicar o movimento retrógrado de Marte

Heliocêntrico (Copérnico / Kepler)



Uma única elipse — sem epíclis

Quem propôs o modelo geocêntrico?

Não foi obra de uma só pessoa — foi sistematizado ao longo de séculos



Eudoxo de Cnido

~360 a.C.

Um dos primeiros a propor um modelo matemático com esferas concêntricas para explicar os movimentos celestes



Aristóteles

384–322 a.C.

Deu a base filosófica do modelo: a Terra como centro imóvel do universo

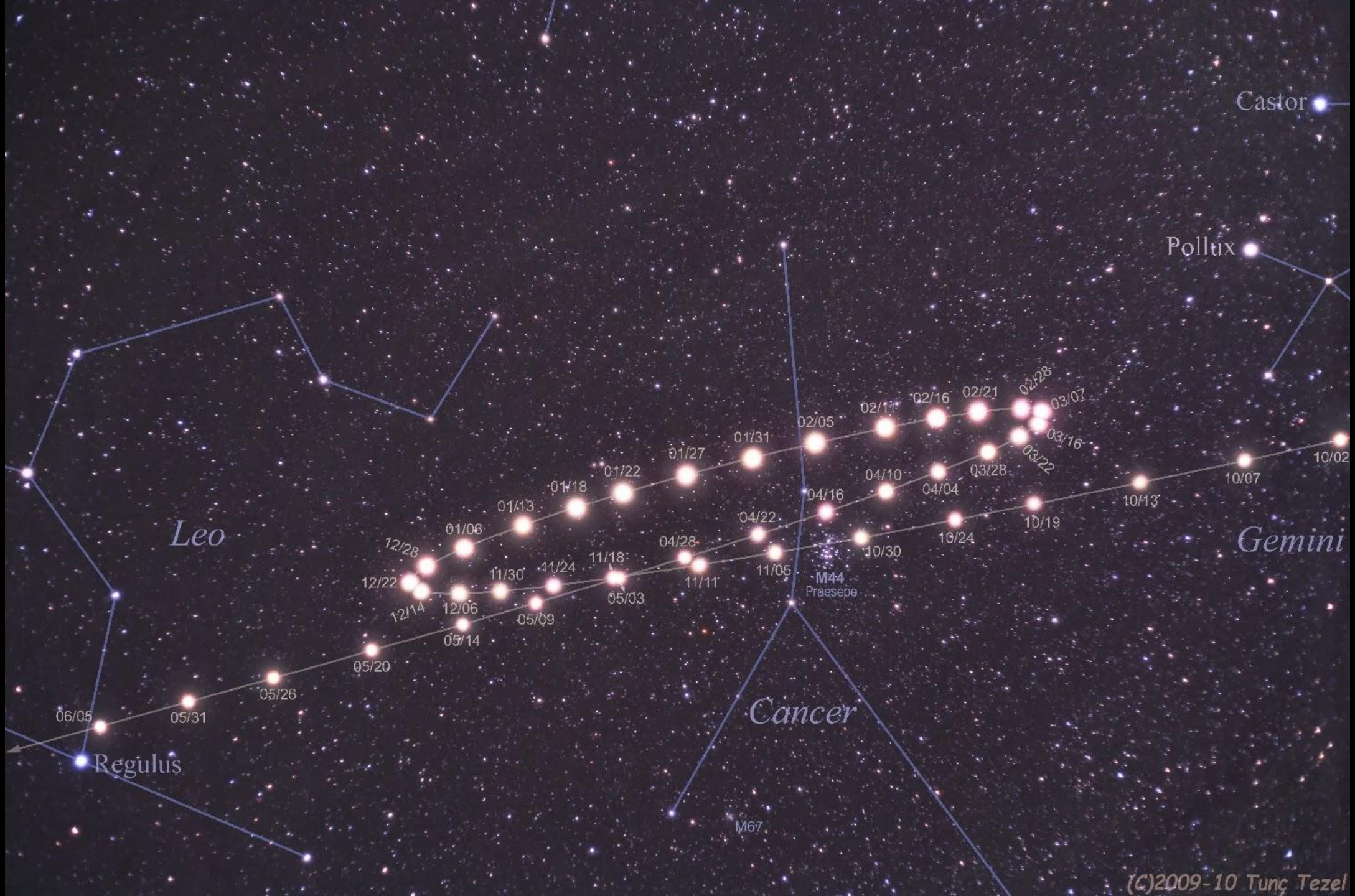


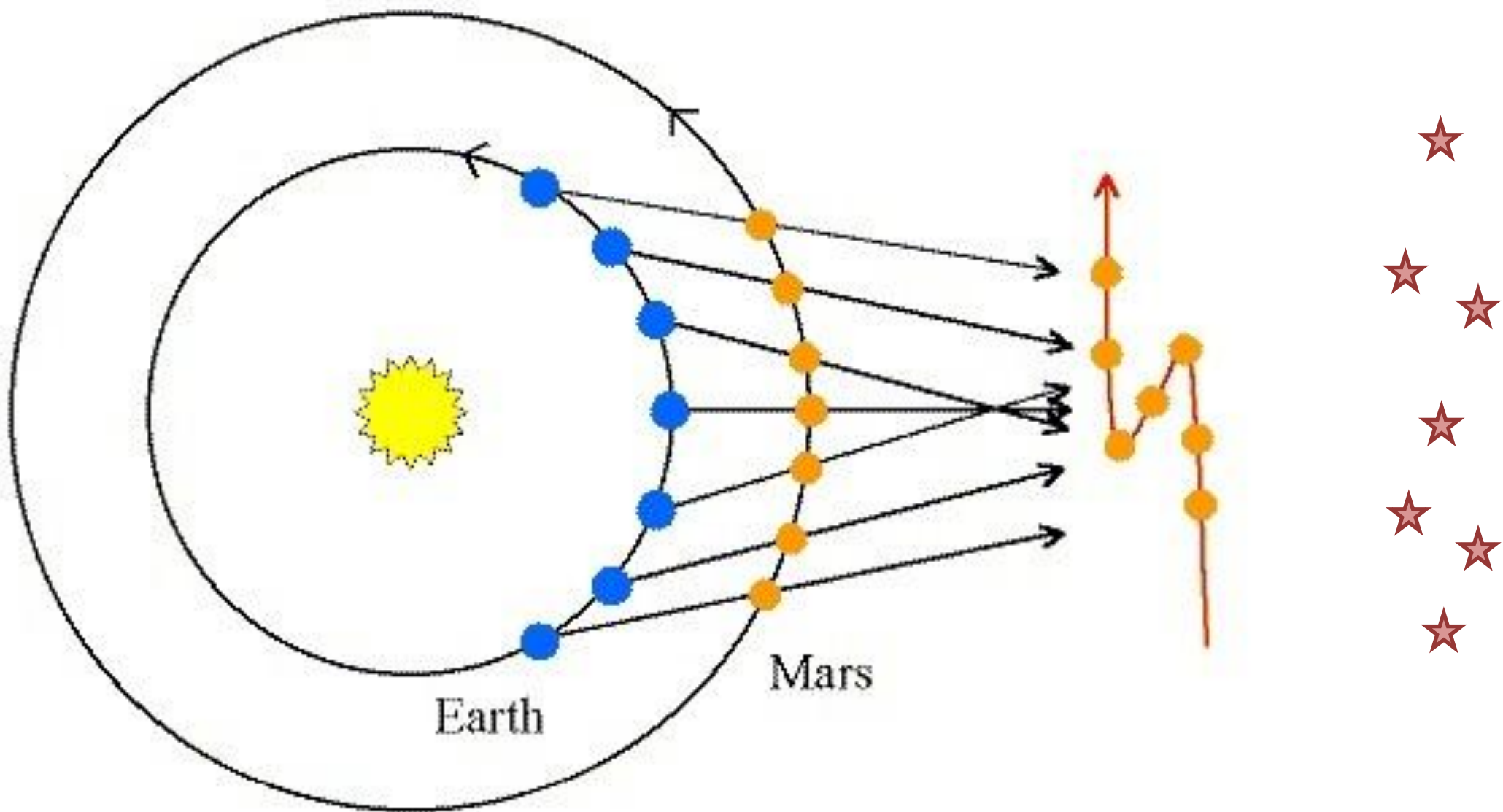
Cláudio Ptolomeu

~150 d.C.

Sistematizou o modelo matematicamente no *Almagesto*, com os epiciclos — por isso “sistema ptolomaico”

Não existem fotografias desses pensadores — a fotografia só foi inventada no século XIX. Bustos e pinturas que os representam são reconstruções muito posteriores.





Quem propôs o modelo heliocêntrico?

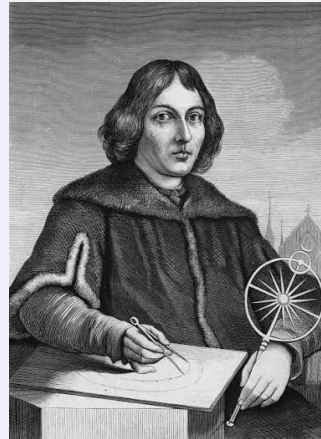
Uma ideia retomada e refinada ao longo de quase 1800 anos



Aristarco de Samos

~310–230 a.C.

Primeiro a propor que o Sol está no centro e a Terra gira ao seu redor — ideia à frente do tempo, esquecida por séculos



Nicolau Copérnico

1473–1543

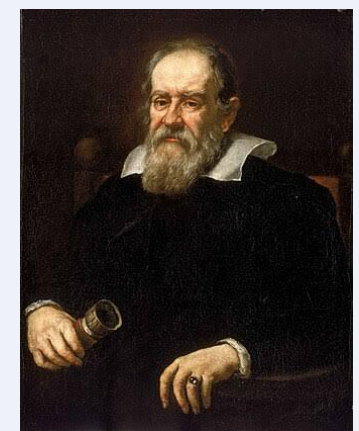
Revive e formaliza o modelo em “De revolutionibus”, publicado em 1543 — por isso “sistema copernicano”



Johannes Kepler

1571–1630

Descobre que as órbitas são elipses, não círculos — elimina a necessidade de epiciclos



Galileu Galilei

1564–1642

Fornece a evidência observacional (luas de Júpiter, fases de Vênus) que ajuda a confirmar o modelo

Aristarco não possui retrato histórico conhecido — apenas Copérnico, Kepler e Galileu têm pinturas/gravuras de época.

Nada no universo mudou. Só mudou o referencial.

A mesma trajetória de Marte que é um emaranhado de loops no referencial da Terra vira uma elipse simples no referencial do Sol.

Geocêntrico × heliocêntrico × galactocêntrico

cada referencial é útil para um tipo de cálculo diferente

GPS do carro

latitude/longitude (referencial da Terra) ou posição relativa ao veículo — mesma posição, duas descrições

O mesmo princípio em Computação Gráfica

Assim como Copérnico escolheu o Sol como origem para simplificar as contas...



Modelar

a origem fica no centro do próprio objeto

ESPAÇO DO OBJETO



Renderizar

a origem fica na câmera — o mundo inteiro se move para o referencial dela

ESPAÇO DA CÂMERA

É por isso que, no pipeline gráfico, a câmera nunca se move de verdade — movemos o mundo inteiro para o referencial dela. Mesma lógica de Copérnico, aplicada a shaders.

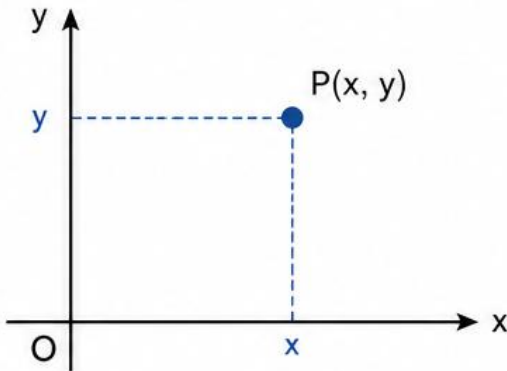
Do objeto ao pixel: uma cadeia de referenciais

Cada etapa reposiciona o mesmo ponto em um novo sistema de coordenadas



COMPARAÇÃO ENTRE SISTEMAS DE COORDENADAS: CARTESIANO 2D vs POLAR

1. CARTESIANO 2D (RETANGULAR)



IDEIA PRINCIPAL

Um ponto P é determinado pelas distâncias horizontal e vertical a partir da origem, medidas ao longo dos eixos perpendiculares x e y.

PARÂMETROS

- x → abscissa: distância algébrica ao longo do eixo x
 y → ordenada: distância algébrica ao longo do eixo y
 $P(x, y)$ → coordenadas cartesianas do ponto

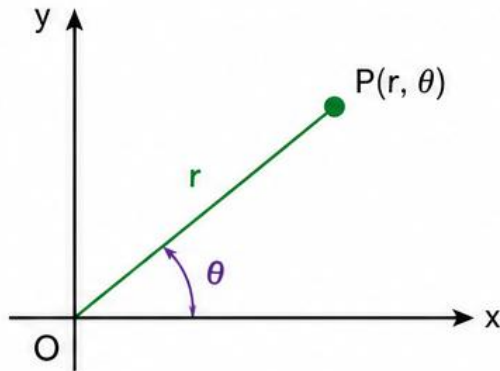
CONVERSÃO PARA POLAR

$$r = \sqrt{x^2 + y^2} \quad r \rightarrow \text{distância da origem ao ponto P } (r \geq 0)$$
$$\theta = \text{atan2}(y, x) \quad \theta \rightarrow \text{ângulo com o eixo x positivo (sentido anti-horário)} \quad (0 \leq \theta < 2\pi)$$

CARACTERÍSTICAS / USO

- Baseado em eixos perpendiculares (x e y).
- Ideal para problemas em geometrias retangulares.
- Muito usado em álgebra, cálculo, gráficos 2D, CAD, física cartesiana.

2. POLAR



IDEIA PRINCIPAL

Um ponto P é determinado pela distância radial r até a origem e pelo ângulo θ em relação ao eixo x positivo.

PARÂMETROS

- r → distância radial da origem até o ponto ($r \geq 0$)
 θ → ângulo medido a partir do eixo x positivo (sentido anti-horário) ($0 \leq \theta < 2\pi$)
 $P(r, \theta)$ → coordenadas polares do ponto

CONVERSÃO PARA CARTESIANO

$$x = r \cos \theta \quad x \rightarrow \text{abscissa (coordenada x)}$$
$$y = r \sin \theta \quad y \rightarrow \text{ordenada (coordenada y)}$$

CARACTERÍSTICAS / USO

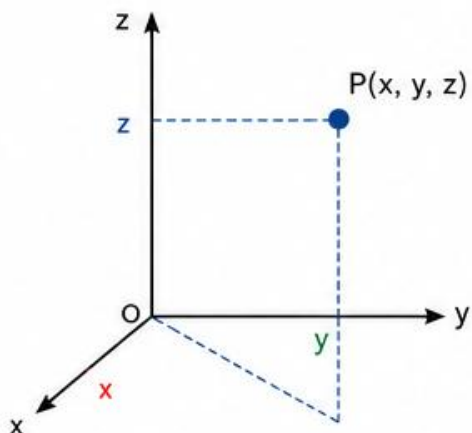
- Baseado em distância e ângulo.
- Ideal para problemas com simetria circular ou radial.
- Muito usado em física, engenharia, ondas, eletromagnetismo, gráficos.

COMPARAÇÃO RESUMIDA

ASPECTO	CARTESIANO 2D (RETANGULAR)		POLAR
Parâmetros	Duas distâncias: (x, y)		Uma distância e um ângulo: (r, theta)
Referência	Eixos perpendiculares x e y		Eixo x positivo (ângulo medido anti-horário)
Intervalos típicos	$x, y \in \mathbb{R}$		$r \in [0, \infty)$ e $\theta \in [0, 2\pi)$
Representação	Linhas retas paralelas aos eixos		Linhas radiais e circunferências concêntricas
Melhor para	Formas retangulares, álgebra, cálculo, gráficos		Simetrias circulares, ondas, campos, integrais duplas

COMPARAÇÃO ENTRE SISTEMAS DE COORDENADAS EM 3D

1. CARTESIANO 3D (RETANGULAR)



IDEIA PRINCIPAL

Um ponto P é determinado pelas distâncias horizontal (x), profundidade (y) e altura (z) a partir da origem O, medidas ao longo dos eixos perpendiculares x, y e z.

PARÂMETROS

- x** → abscissa: distância algébrica ao longo do eixo x
y → ordenada: distância algébrica ao longo do eixo y
z → cota (altura): distância algébrica ao longo do eixo z
P(x, y, z) → coordenadas cartesianas do ponto

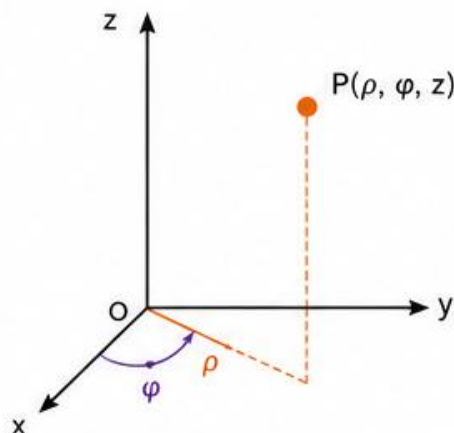
VOLUME DE UM ELEMENTO

$$dV = dx \, dy \, dz$$

CARACTERÍSTICAS / USO

- Baseado em três eixos perpendiculares (x, y, z).
- Ideal para problemas sem simetria específica.
- Muito usado em álgebra vetorial, CAD, física cartesiana, computação gráfica 3D.

2. CILÍNDRICO



IDEIA PRINCIPAL

Um ponto P é determinado pela distância radial ρ até o eixo z, pelo ângulo φ no plano xy (a partir do eixo x positivo) e pela altura z.

PARÂMETROS

- ρ** → distância radial da projeção de P no plano xy até a origem ($\rho \geq 0$)
 φ → ângulo no plano xy a partir do eixo x positivo (sentido anti-horário) ($0 \leq \varphi < 2\pi$)
z → altura (distância algébrica ao plano xy)
P(ρ, φ, z) → coordenadas cilíndricas do ponto

CONVERSÃO PARA CARTESIANO

$$\begin{aligned}x &= \rho \cos \varphi \\y &= \rho \sin \varphi \\z &= z\end{aligned}$$

CONVERSÃO PARA CILÍNDRICO

$$\begin{aligned}\rho &= \sqrt{x^2 + y^2} \\ \varphi &= \text{atan2}(y, x) \quad (0 \leq \varphi < 2\pi) \\ z &= z\end{aligned}$$

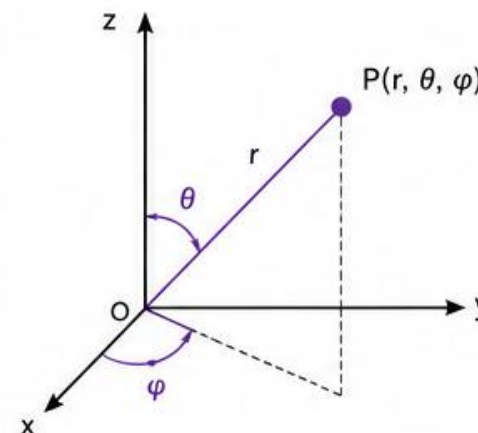
VOLUME DE UM ELEMENTO

$$dV = \rho \, d\rho \, d\varphi \, dz$$

CARACTERÍSTICAS / USO

- Baseado em simetria cilíndrica (em torno do eixo z).
- Ideal para problemas com formas cilíndricas, tubos, fios retilíneos, escoamentos axiais, campos magnéticos.

3. ESFÉRICO



IDEIA PRINCIPAL

Um ponto P é determinado pela distância radial r até a origem, pelo ângulo θ (inclinação em relação ao eixo z positivo) e pelo ângulo φ (azimute no plano xy a partir do eixo x positivo).

PARÂMETROS

- r** → distância radial até a origem ($r \geq 0$)
 θ → ângulo de inclinação a partir do eixo z positivo ($0 \leq \theta \leq \pi$)
 φ → ângulo azimutal no plano xy a partir do eixo x positivo ($0 \leq \varphi < 2\pi$)
P(r, θ, φ) → coordenadas esféricas do ponto

CONVERSÃO PARA CARTESIANO

$$\begin{aligned}x &= r \sin \theta \cos \varphi \\y &= r \sin \theta \sin \varphi \\z &= r \cos \theta\end{aligned}$$

CONVERSÃO PARA ESFÉRICO

$$\begin{aligned}r &= \sqrt{x^2 + y^2 + z^2} \\ \theta &= \arccos\left(\frac{z}{r}\right) \quad (0 \leq \theta \leq \pi) \\ \varphi &= \text{atan2}(y, x) \quad (0 \leq \varphi < 2\pi)\end{aligned}$$

VOLUME DE UM ELEMENTO

$$dV = r^2 \sin \theta \, dr \, d\theta \, d\varphi$$

CARACTERÍSTICAS / USO

- Baseado em simetria esférica (em torno da origem).
- Ideal para problemas com esferas, cascas esféricas, ondas esféricas, gravitação, radiação.

COMPARAÇÃO RESUMIDA			
ASPECTO	CARTESIANO 3D	CILÍNDRICO	ESFÉRICO
Parâmetros	Três distâncias: (x, y, z)	Duas distâncias e um ângulo: (ρ, ϕ, z)	Uma distância e dois ângulos: (r, θ, ϕ)
Faixas típicas	$x, y, z \in \mathbb{R}$	$\rho \in [0, \infty), \phi \in [0, 2\pi), z \in \mathbb{R}$	$r \in [0, \infty), \theta \in [0, \pi], \phi \in [0, 2\pi)$
Elementos de volume	$dV = dx \, dy \, dz$	$dV = \rho \, d\rho \, d\phi \, dz$	$dV = r^2 \sin \theta \, dr \, d\theta \, d\phi$
Melhor para	Geometrias gerais, problemas sem simetria	Simetria cilíndrica (em torno do eixo z)	Simetria esférica (em torno da origem)
Exemplo de aplicação	Modelagem 3D geral, física cartesiana, computação gráfica	Escoamento em dutos, campos magnéticos em fios, problemas de calor em cilindros	Gravitação, radiação, ondas esféricas, átomos (orbitais), acústica

LEGENDA DOS SÍMBOLOS

x, y, z

: eixos cartesianos

ρ

: distância radial no plano xy (cilíndrico)

ϕ

: ângulo azimutal no plano xy (a partir do eixo x positivo)

z

: altura (cilíndrico)

r

: distância radial até a origem (esférico)

θ

: ângulo de inclinação a partir do eixo z positivo (esférico)

Coordenadas polares, cilíndricas e esféricas em CG

Cada sistema simplifica uma classe diferente de problema

Sistema	Aplicação típica em Computação Gráfica
Polares	Texturas radiais, gradientes, algoritmos de preenchimento circular
Cilíndricas	Objetos com simetria axial (canos, garrafas, colunas), mapeamento de textura cilíndrico
Esféricas	Orbit camera, posicionamento de luzes direcionais/ambientais, mapeamento de textura esférico (skyboxes), geração procedural de terrenos e planetas

Exercício rápido: calcule a posição cartesiana de uma câmera orbital dado um raio fixo e ângulos (ϑ, φ) — conecta direto com a View Matrix.

Cartesian coordinate

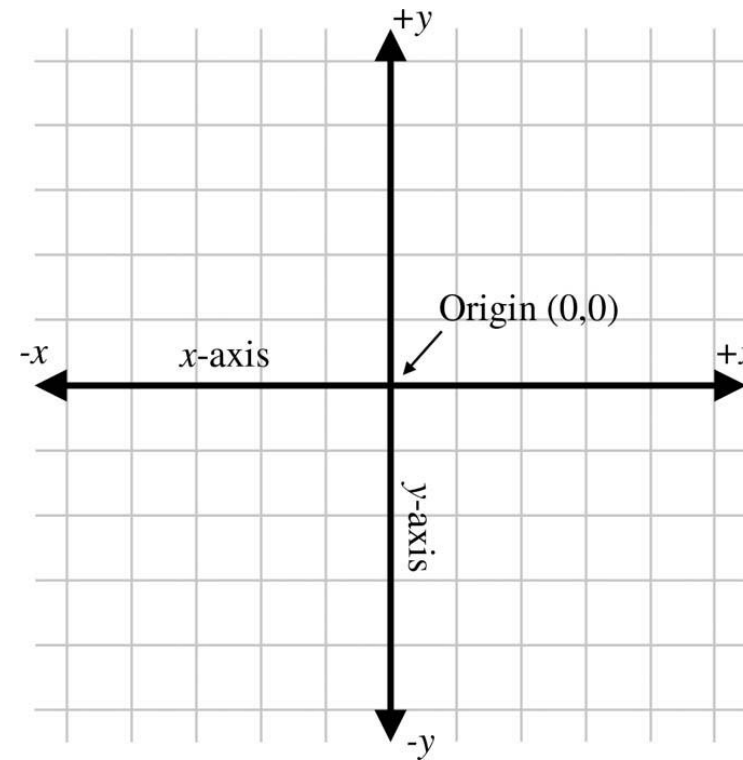
- 3D math is all about measuring locations, distances, and angles precisely and mathematically in 3D space.
- The most frequently used framework to perform such calculations using a computer is called the *Cartesian coordinate system*.

1D Mathematics

- We assume that you already know about the natural numbers, the integers, the rational numbers, and the real numbers.
- On a computer you have to make do with **shorts**, **ints**, **floats**, and **doubles**. These have limited precision.
- We assume that you have a basic understanding about how numbers are represented on a computer.
- Remember the First Law of Computer Graphics:
If it looks right, it is right.

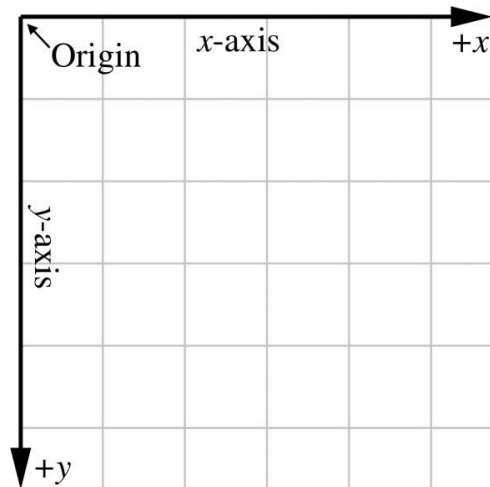
2D Coordinate Spaces

- All that really matters are the numbers.
- The abstract version of this is called a *2D Cartesian coordinate space*.



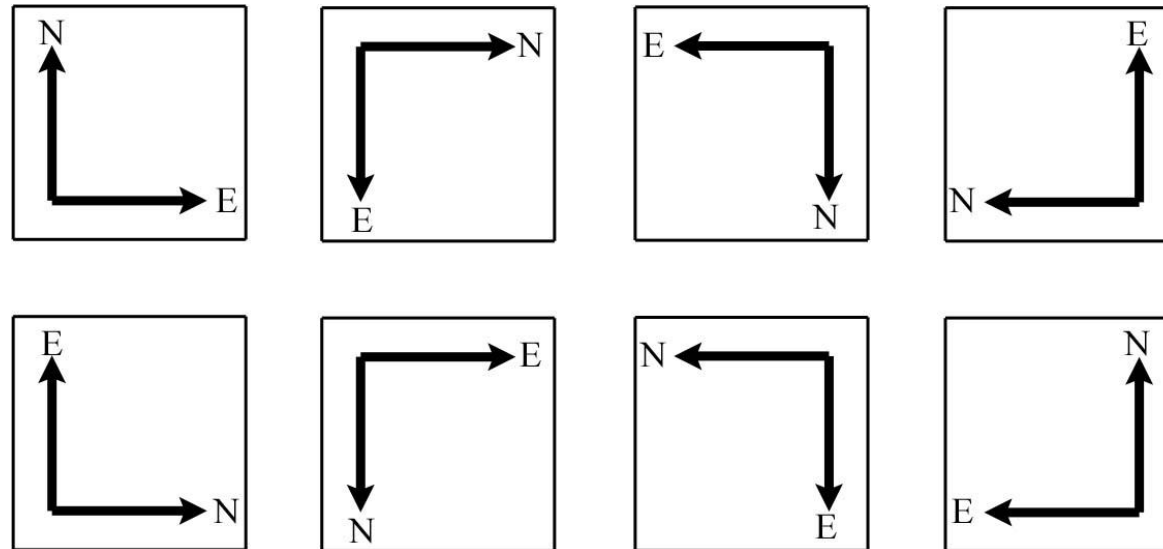
Screen Space

- But it doesn't *have to* be this way. It's only a convention.
- In screen space, for example, $+y$ points down.
- Screen space is how you measure on a computer screen, with the origin at the top left corner.



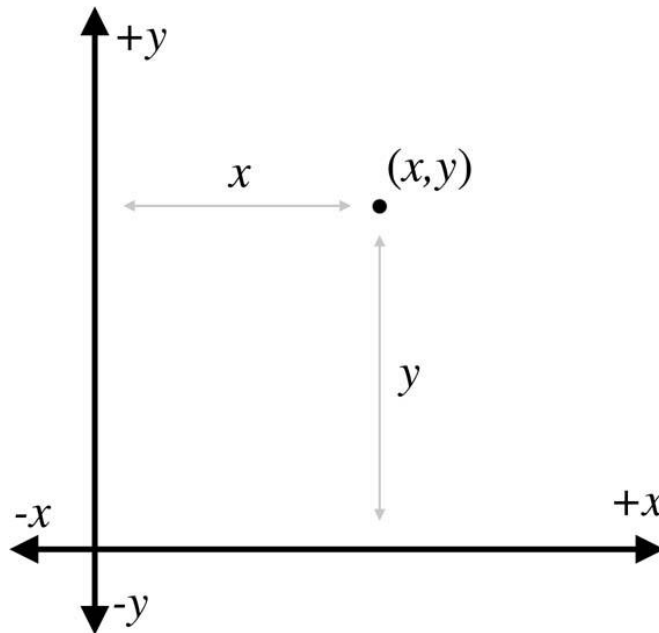
Axis Orientation

There are 8 possible ways of orienting the Cartesian axes.

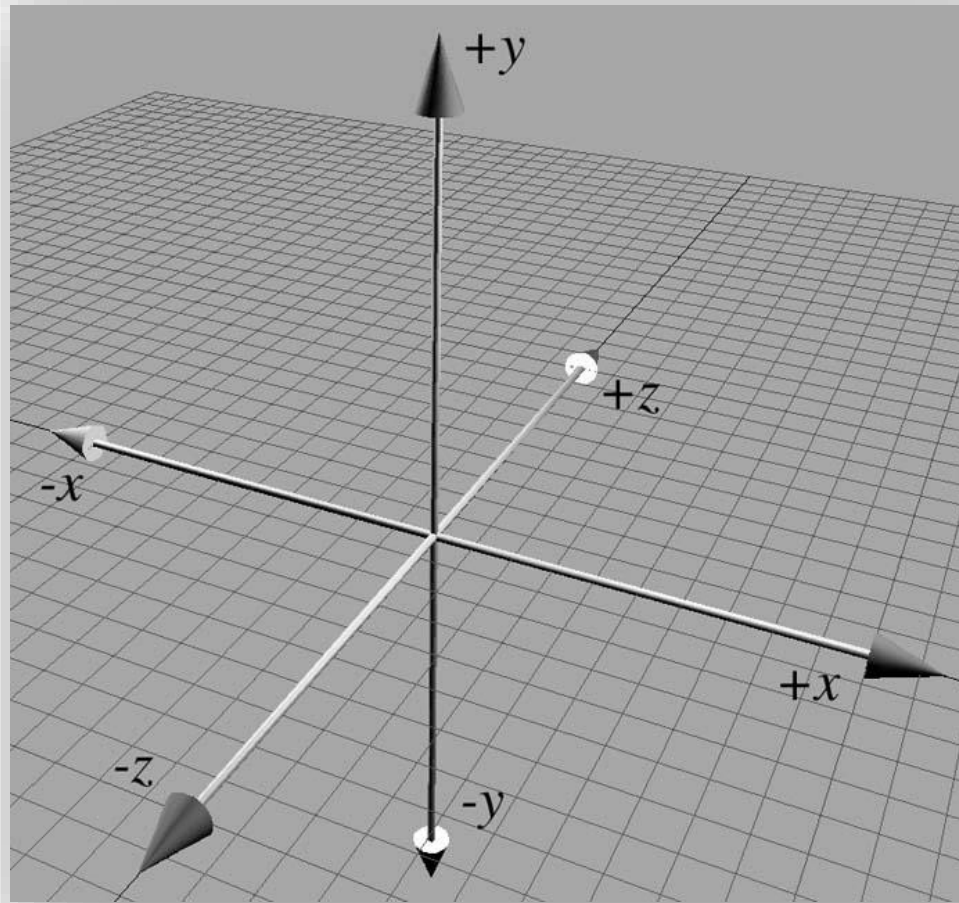


Locating Points in 2D

Point (x,y) is located x units across and y units up from the origin.

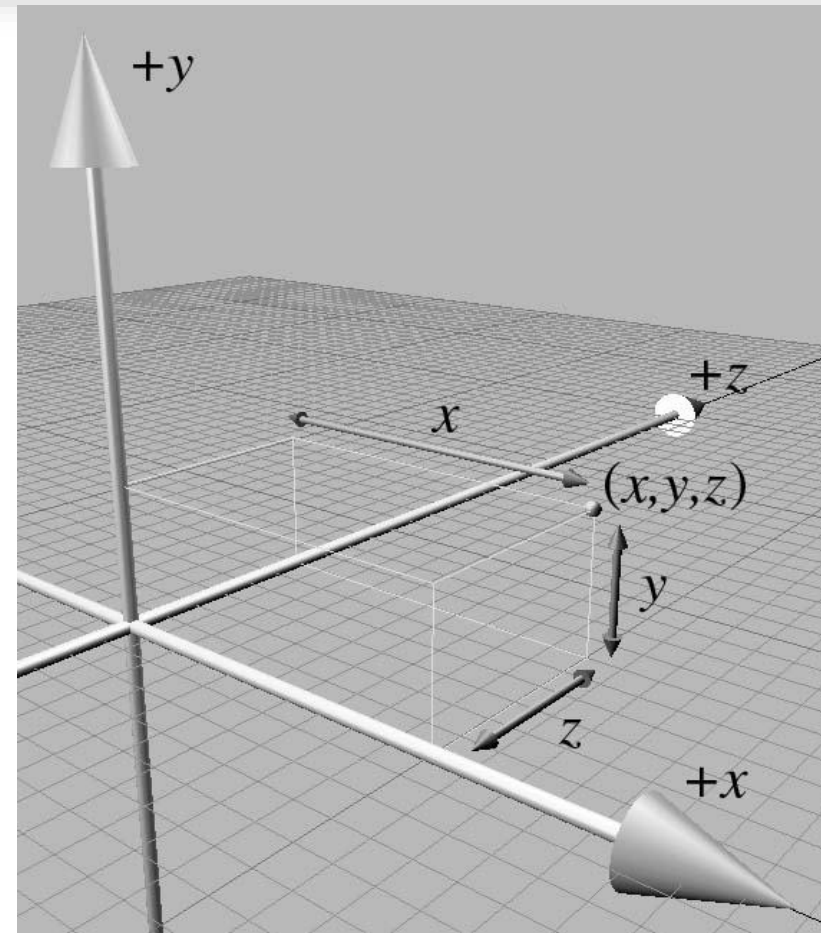


3D Cartesian Space



Locating Points in 3D

Point (x,y,z) is located x units along the x -axis, y units along the y -axis, and z units along the z -axis from the origin.



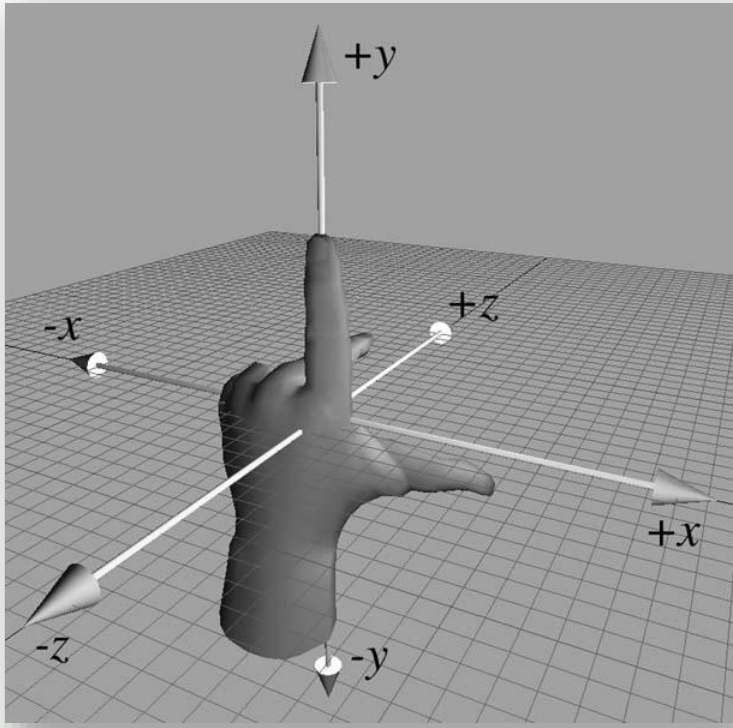
Axis Equivalence in 3D

- Recall that in 2D the alternatives for axis orientation can be obtained by rotating the map around 2 axes.
- As we said, this is not true of 3D coordinate space.

Visualizing 3D Space

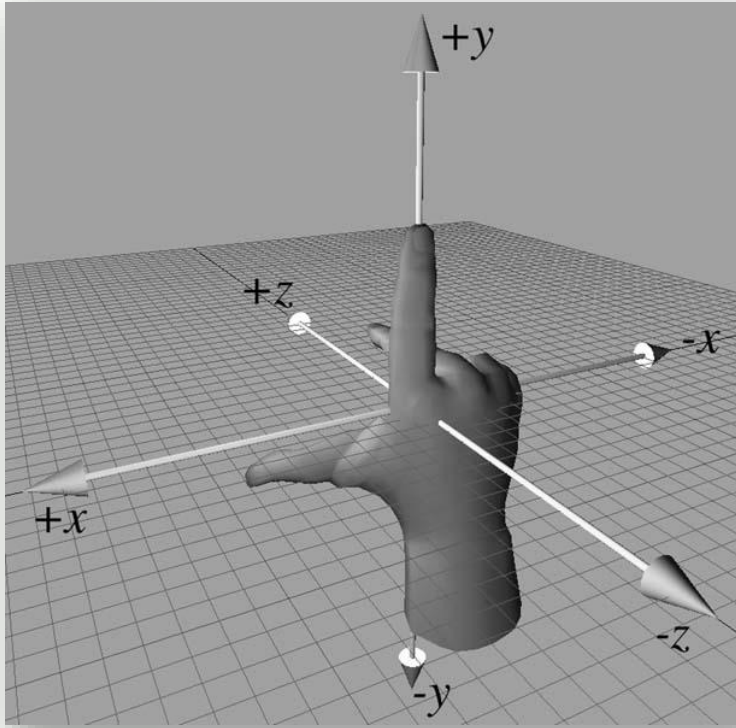
- The usual convention is that the x -axis is horizontal and positive is right, and that the y -axis is vertical and positive is up
- The z -axis is depth, but should the positive direction go forwards “into” the screen or backwards “out from” the screen?

Left-handed Coordinates



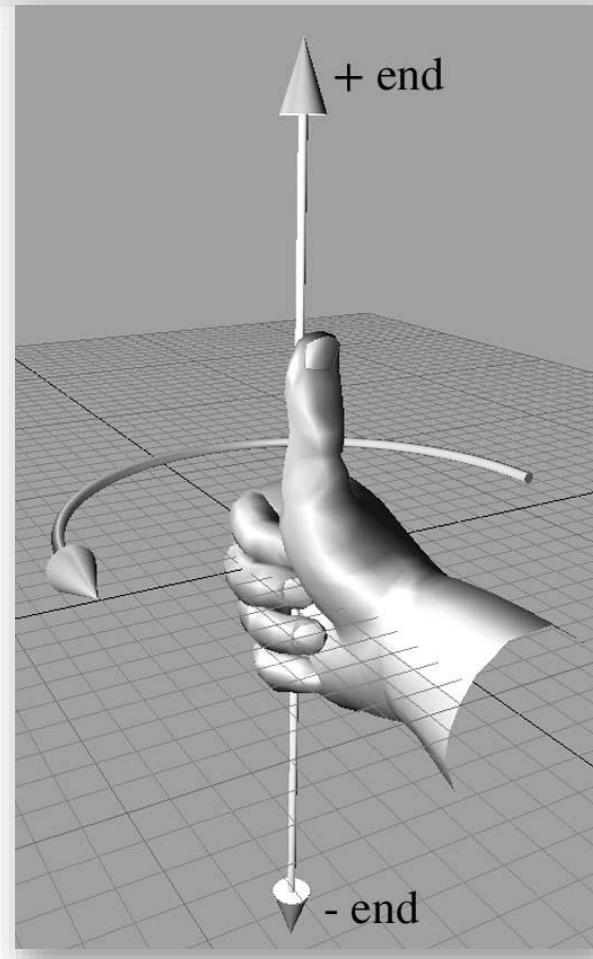
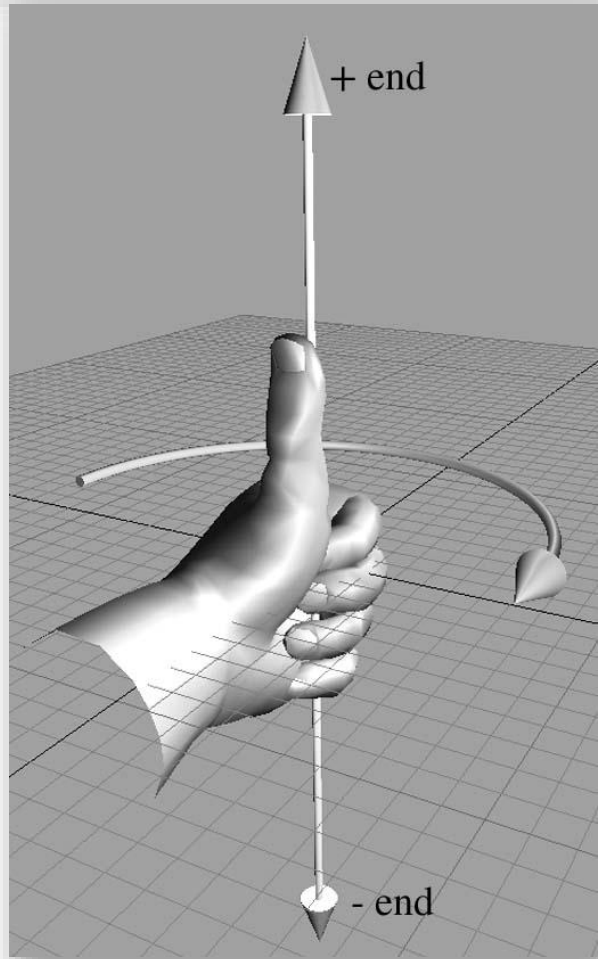
- $+z$ goes “into” screen
- Use your left hand
- Thumb is $+x$
- Index finger is $+y$
- Second finger is $+z$

Right-handed Coordinates



- $+z$ goes “out from” screen
- Use your right hand
- Thumb is $+x$
- Index finger is $+y$
- Second finger is $+z$
- (Same fingers, different hand)

Positive Rotation



O pipeline gráfico: sete espaços, seis matrizes

Cada seta é uma multiplicação de matriz aplicada ao mesmo ponto

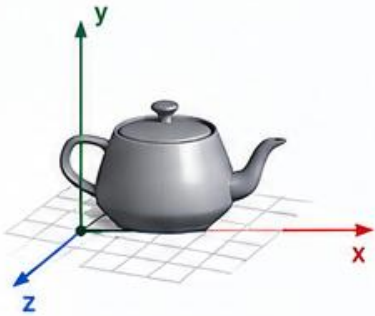


+ espaço de textura (UV) — sistema paralelo para mapear texturas sobre a superfície

SISTEMAS DE REFERÊNCIA NA PIPELINE GRÁFICA

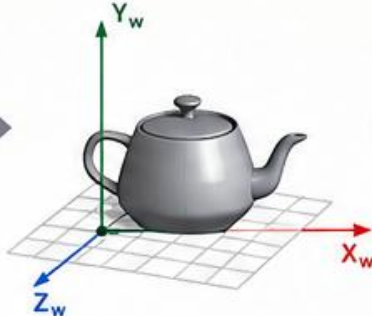
Transformação de um objeto 3D até sua exibição 2D na tela

1 Espaço do Modelo (Model Space)



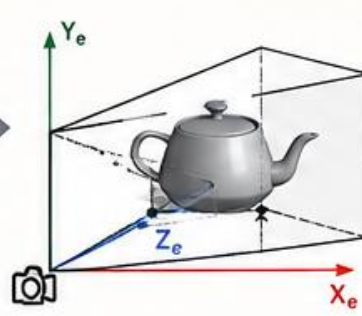
- Coordenadas definidas pelo autor do modelo.
- Origem e orientação arbitrárias.
- Unidade: unidades do mundo do modelo.

2 Espaço do Mundo (World Space)



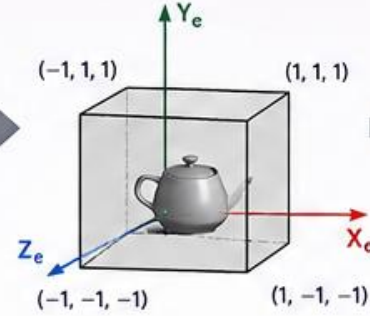
- Posiciona o(s) objeto(s) na cena.
- Define a relação com outros objetos e com a câmera.
- Unidade: unidades do mundo.

3 Espaço da Câmera / Visão (View / Eye Space)



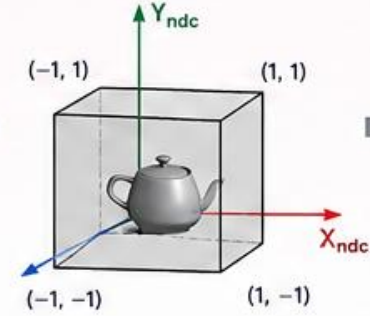
- Obtido pela transformação de visão ("look-at").
- Câmera na origem olhando para $-Z_e$.
- Define o que a câmera vê.

4 Espaço de Recorte (Clip Space)



- Resultado da projeção (perspectiva ou ortográfica).
- Volume padrão (cubo):
 $-1 \leq x_c, y_c, z_c \leq 1$
- Pronto para o recorte (clipping).

5 Espaço Normalizado do Dispositivo (Normalized Device Coordinates - NDC)



- Após o clipping, permanece no intervalo padrão.
- Independente da resolução da tela.
- Usado para o mapeamento para a tela.

6 Espaço da Tela (Screen Space)



- Coordenadas em pixels.
- Origem no canto superior esquerdo (convencional).
- Intervalos: $0 \leq x_s < W$
 $0 \leq y_s < H$

SISTEMAS DE REFERÊNCIA NA PIPELINE GRÁFICA

Transformação de um objeto 3D até sua exibição 2D na tela

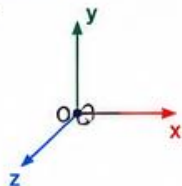
TRANSFORMAÇÕES ENTRE OS ESPAÇOS



1 Modelo (Model Space)

Coordenadas do objeto como foi modelado.

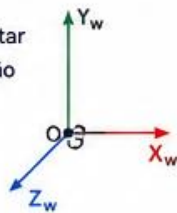
Ex.: unidades em metros, centímetros, etc.



2 Mundo (World Space)

Todos os objetos posicionados no mundo comum.

Permite movimentar objetos em relação uns aos outros.

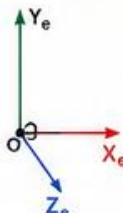


3 Visão (View / Eye Space)

A câmera está na origem. Define o ponto de vista.

Eixos da câmera:

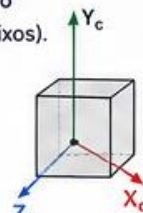
- X_e : direita
- Y_e : cima
- Z_e : para trás ($-Z_e$ é a direção do olhar)



4 Clip Space

Após a projeção.

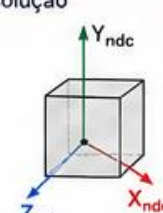
Volume visível mapeado para o cubo padrão (-1 a 1 em todos eixos).



5 NDC

Coordenadas normalizadas após o clipping.

Independente da resolução do dispositivo.

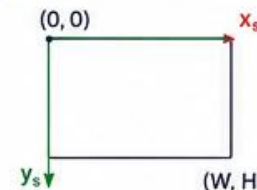


6 Tela (Screen Space)

Coordenadas em pixels para exibição.

Origem (0,0) no canto superior esquerdo.

x_s → direita
 y_s → baixo



Observação: Todas as coordenadas na pipeline são representadas em coordenadas homogêneas (x, y, z, w), permitindo que transformações lineares (matrizes 4×4) representem rotações, translações, projeções e divisões de perspectiva de forma uniforme.

World Space

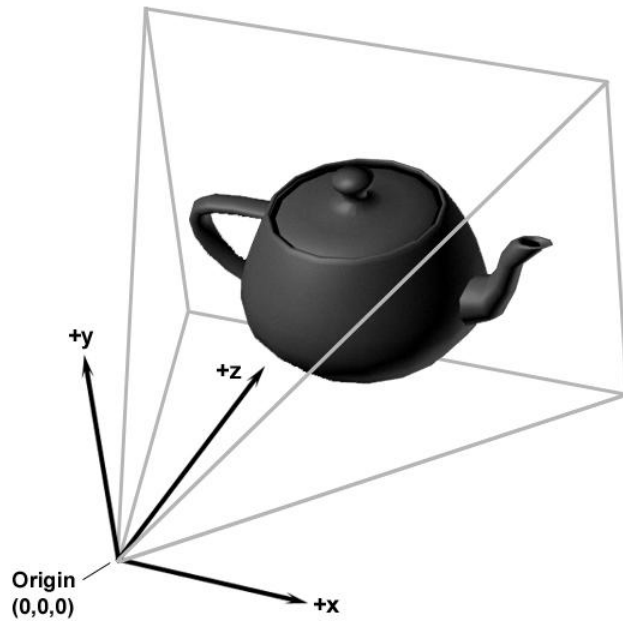
- World space is the global coordinate system.
- Use it to keep track of position and orientation of every object in the game.
- Just like in the movie *Highlander*, there can be only one.

Object Space

Every object in the game has:

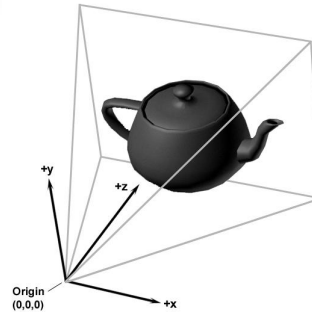
- Its own origin (where it is),
- Its own concept of “up” and “right” and “forwards”,
- That is, its own coordinate space.
- Use it to keep track of relative positions and orientation (eg. Collision detection, AI)

Camera Space



Object space for the viewer, represented by a camera, used to project 3D space onto screen space

Camera Space Terminology

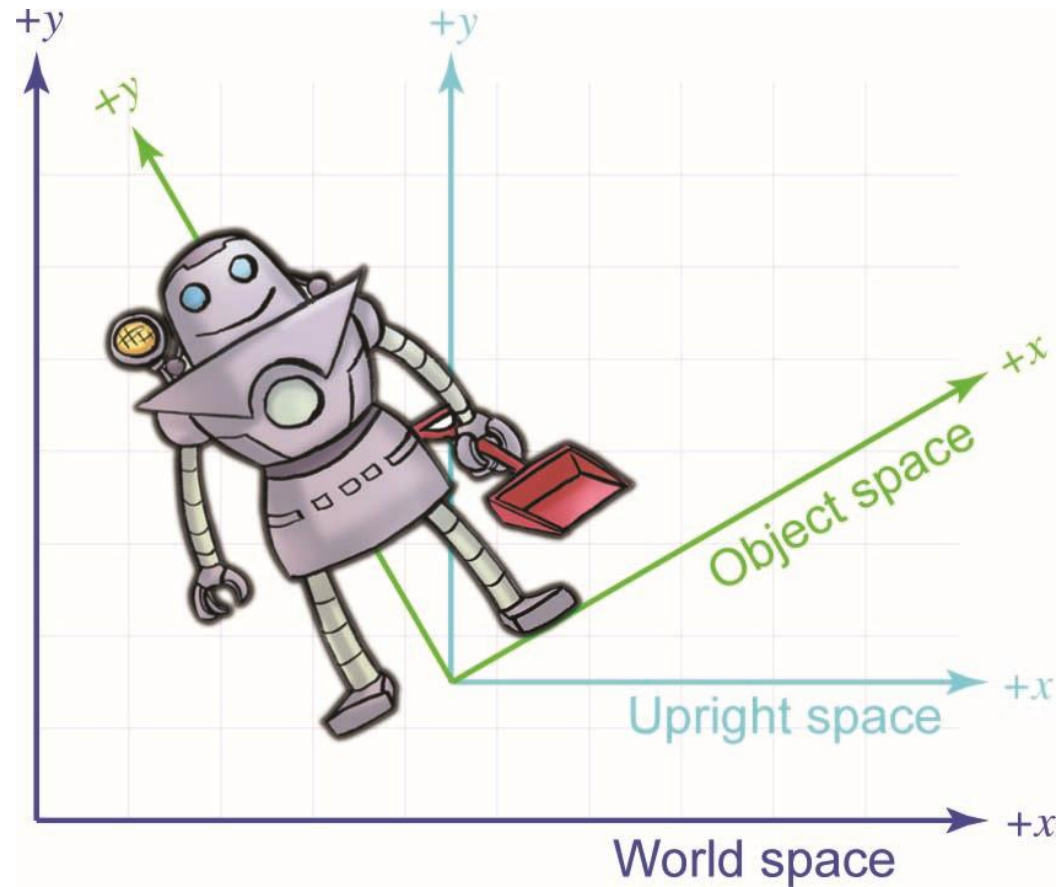


- *Frustum*: the pyramid of space seen by the camera
- *Clipping*: objects partially on screen
- *Occlusion*: objects hidden behind another object
- *Visibility*: inside or outside of frustum, occluded, clipped

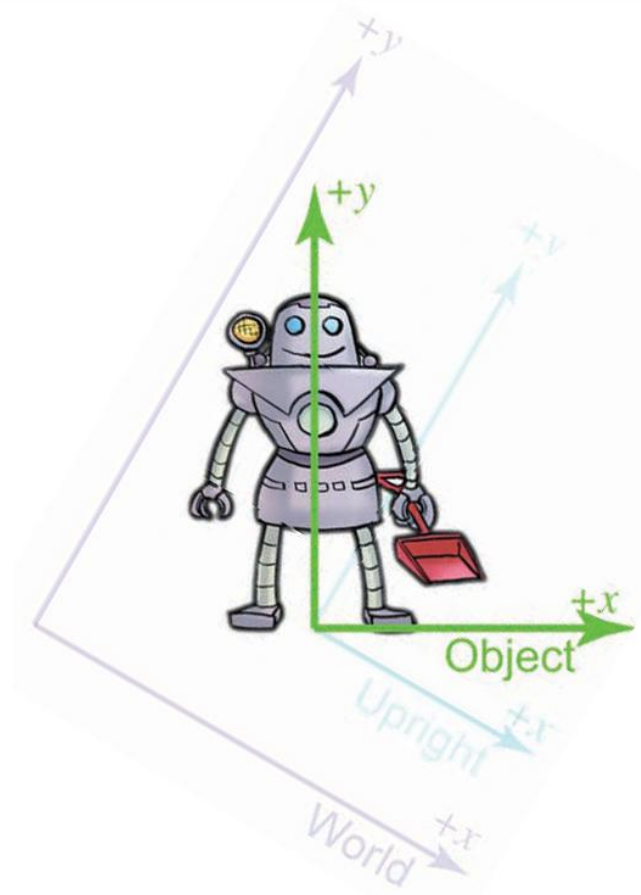
Upright Space

- Upright space is in a sense “half way” between world space and object space.
- Upright space has
 - Object space origin
 - World space axes
- It is nonstandard, we use it in our book because we like it.

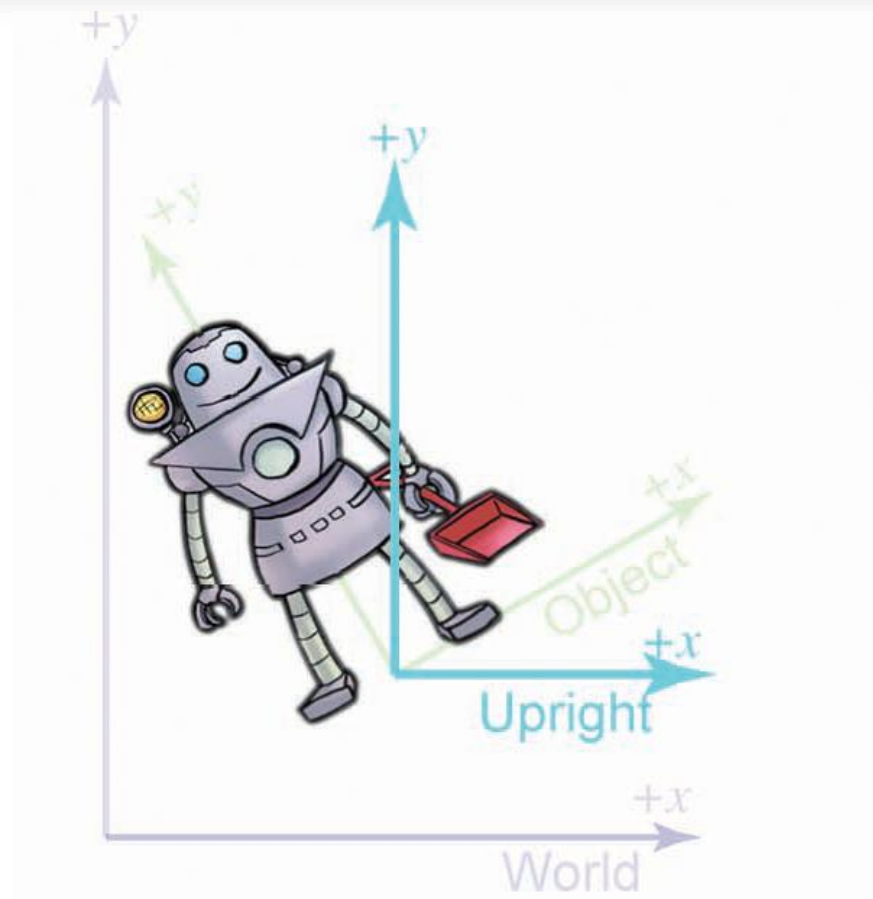
Object, Upright, and World Space



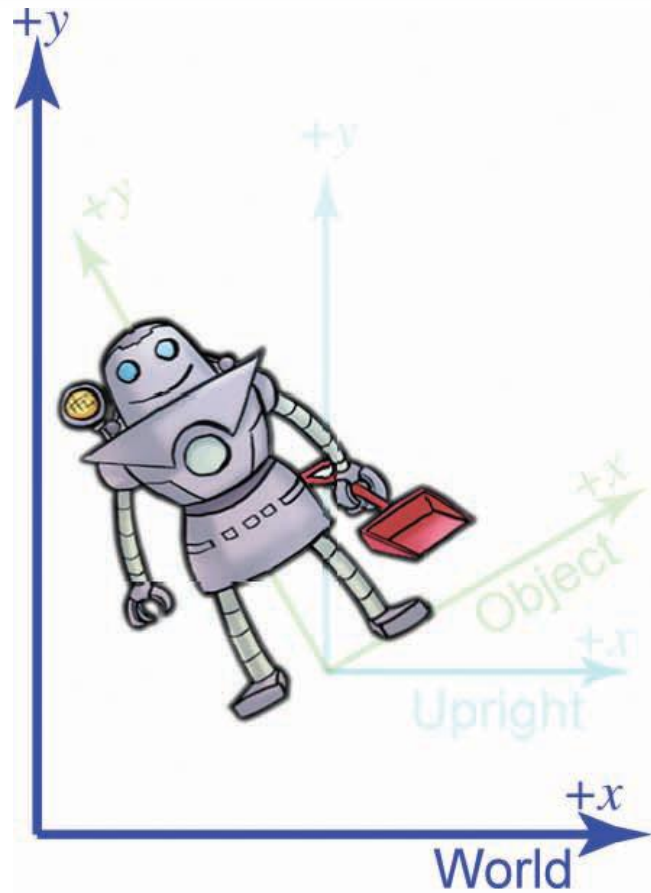
Robot's Object Space



Robot's Upright Space



Robot in World Space



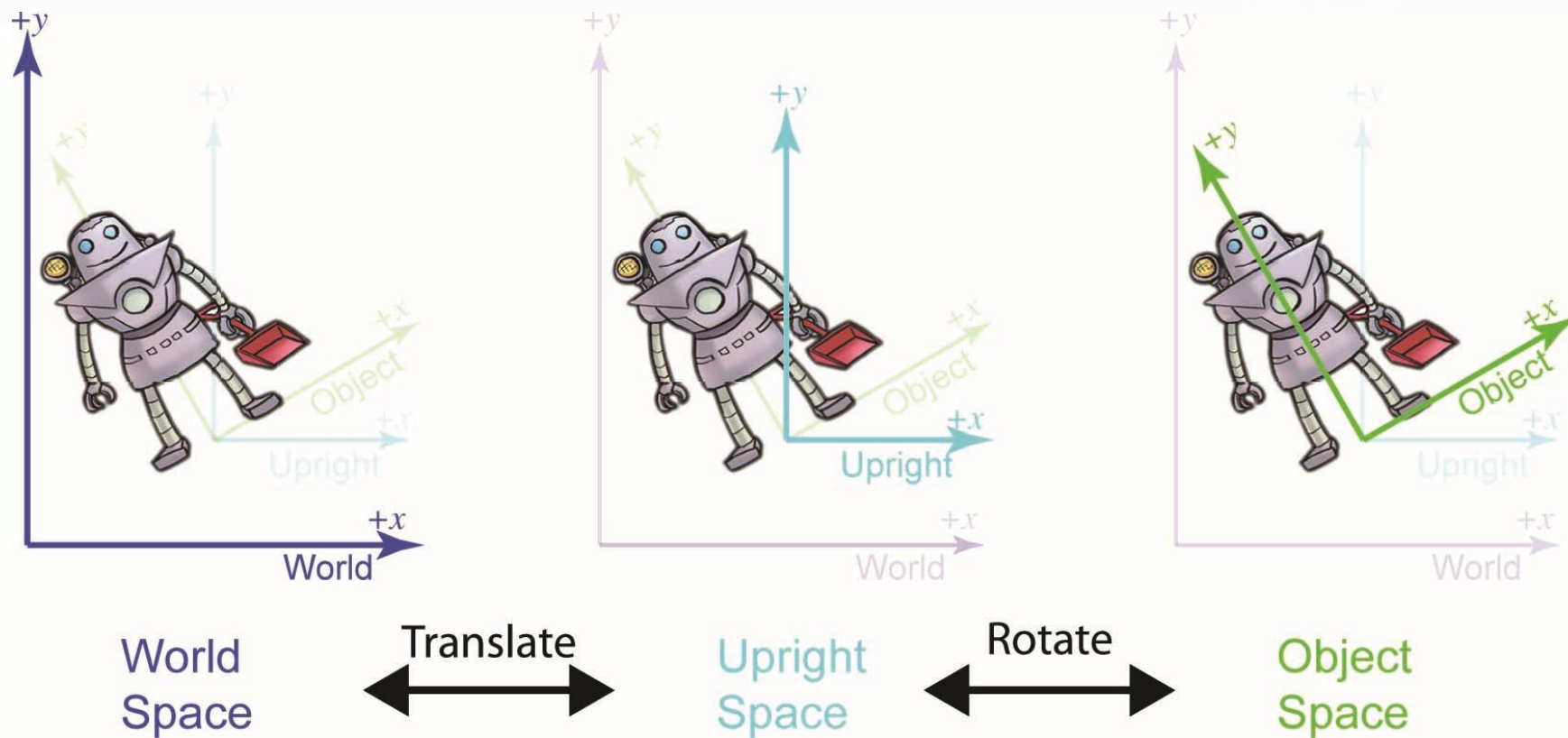
Why Upright Space?

- It separates translation and rotation
 - It is a handy visualization tool
 - It is inspired by both math and hardware implementation
- Translate between world and upright space.
- Rotate between upright and object space.
- Which brings us to *coordinate space transformations...*

Coordinate Space Transformation

Two important types of transformation used to transform one coordinate space into another:

- Translation
 - Changes position in space
 - Gives new location of origin
- Rotation
 - Changes orientation in space
 - Gives new directions of axes



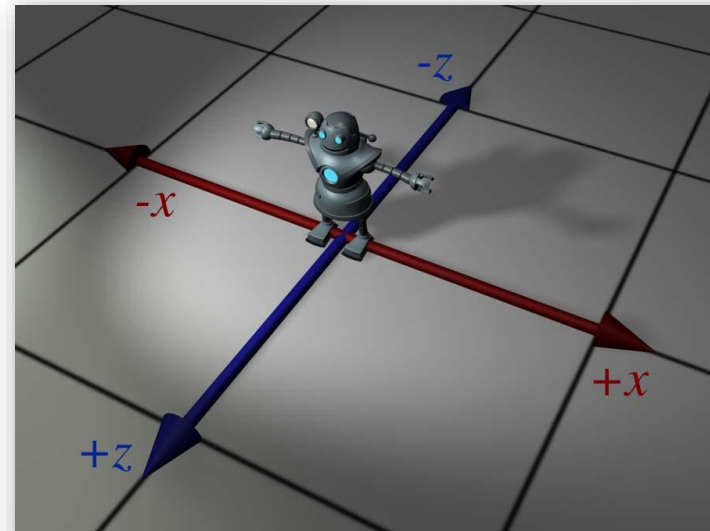
Example

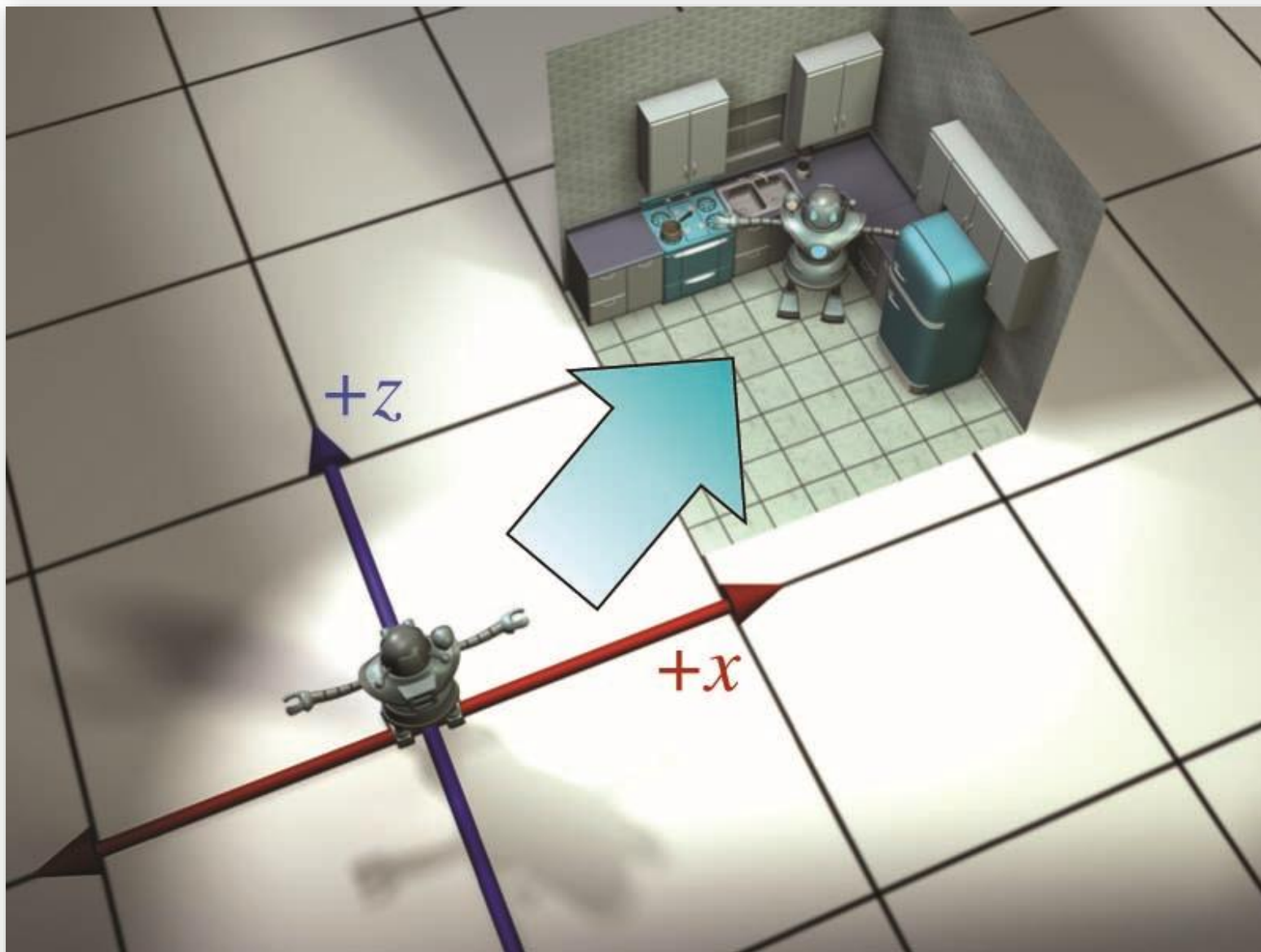


- Let's say that we are working for a advertising agency that has just landed a big account with a food manufacturer.
- You are assigned to the project to make a slick computer-generated ad promoting one of their most popular items, *Herring Packets*, which are microwaveable herring sandwiches food products for robots.
- Of course, the client has a tendency to want changes made at the last minute, so we need to be able to get a model of the product at any possible position and orientation.

Start with the Artist's Model

- For now, because we have the model in its home position, object space and world space (and upright space) are all the same by definition.
- For all practical purposes, in the scene that the artist built containing only the model of the robot, *world space is object space*.

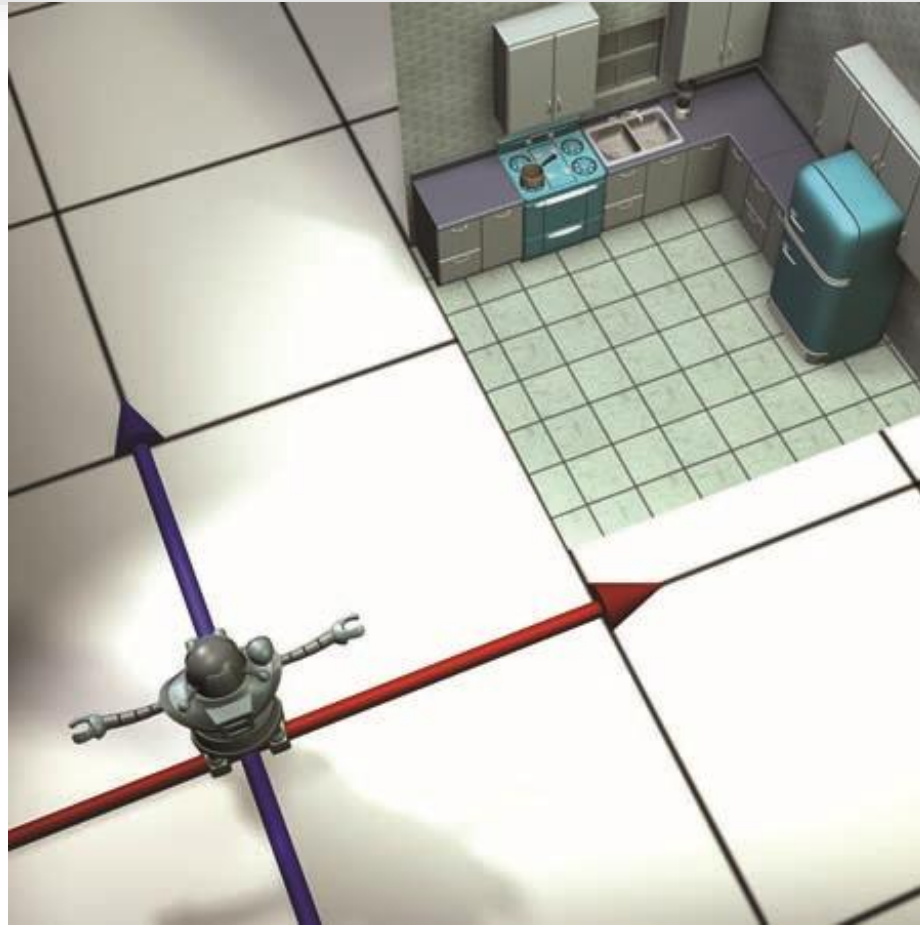




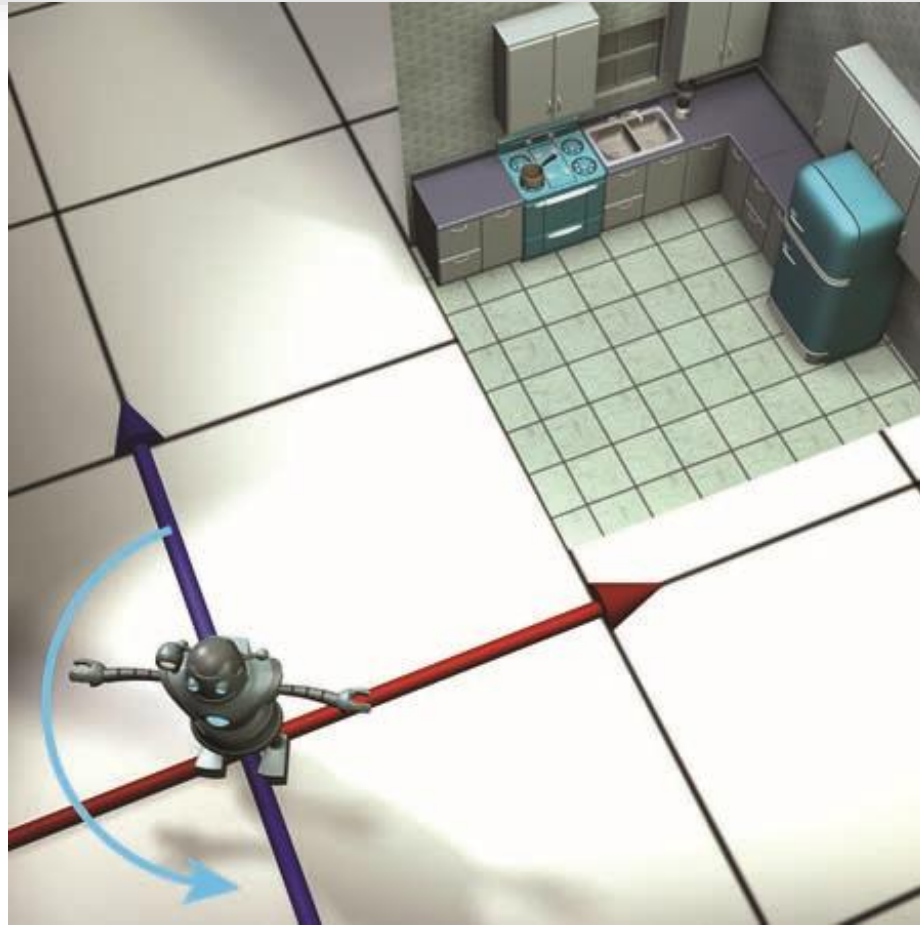
More Details

- Let's talk a bit about how to accomplish this. We won't get too far into the mathematical details - that's what the rest of this chapter is for.
- Conceptually, to move the robot into position we first rotate her clockwise 120° (or, as we'll learn in Chapter 8, by “heading left 120° ”).
- Then we translate 18ft east and 10ft north, which according to our conventions is a 3D displacement of $[18, 0, 10]$.

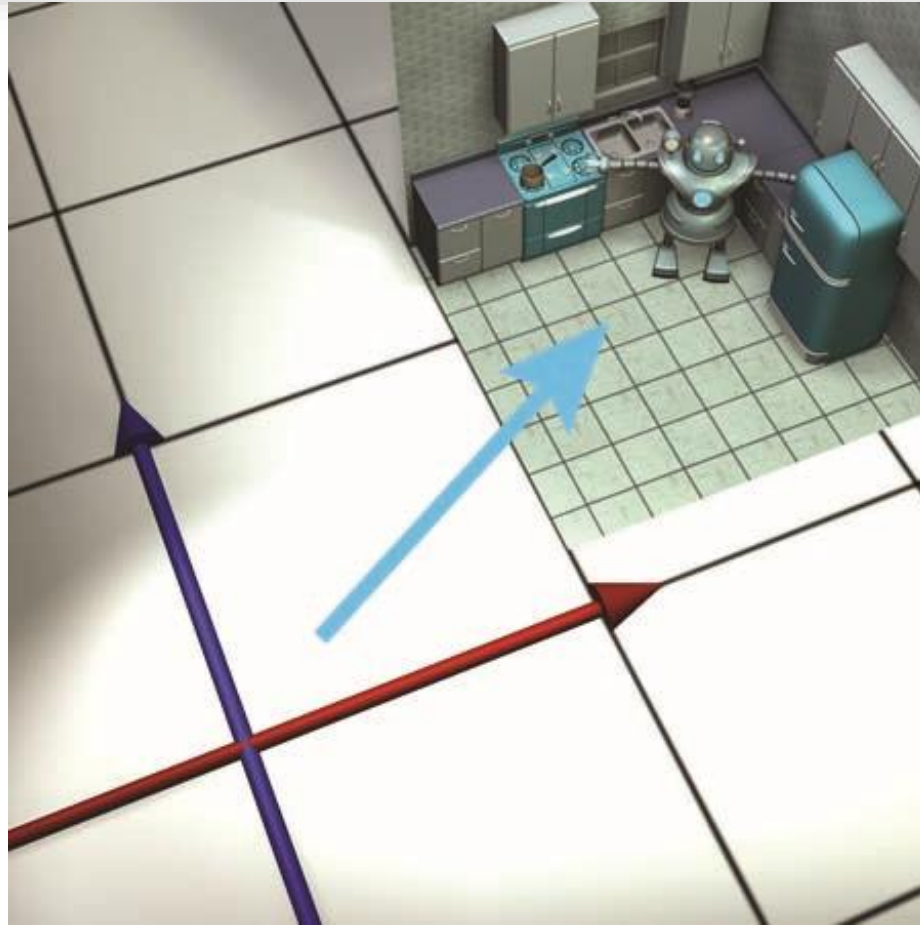
Original Position



Rotate



Then Translate

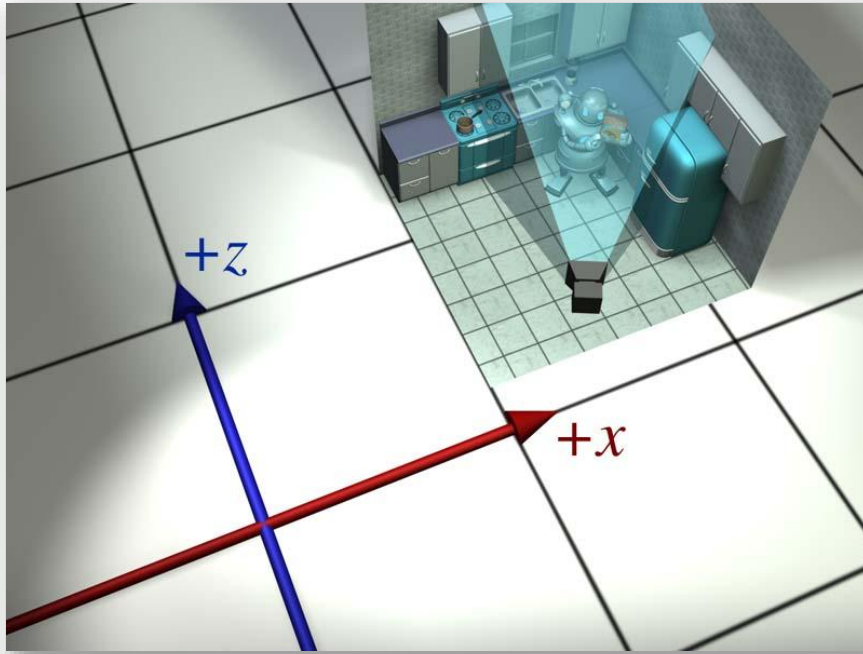


Rotate Before Translate

- Why rotate before we translate?
- Rotation about the origin is easier – translating first would mean that we have to rotate around an arbitrary point.
- Rotation about the origin is a linear transform.
- Rotation about an arbitrary point is an affine transform.
- Affine transforms can be expressed as a sequence of primitive operations.

Camera Space

- So we've managed to get the robot model into the right place in the world.
- But to render it, we need to transform the vertices of the model into *camera space*.
- In other words, we need to express the vertices' coordinates relative to the camera. For example, if a vertex is 9ft in front of the camera and 3ft to the right, then the z and x coordinates of that vertex in camera space would be 9 and 3, respectively.



Where the camera is

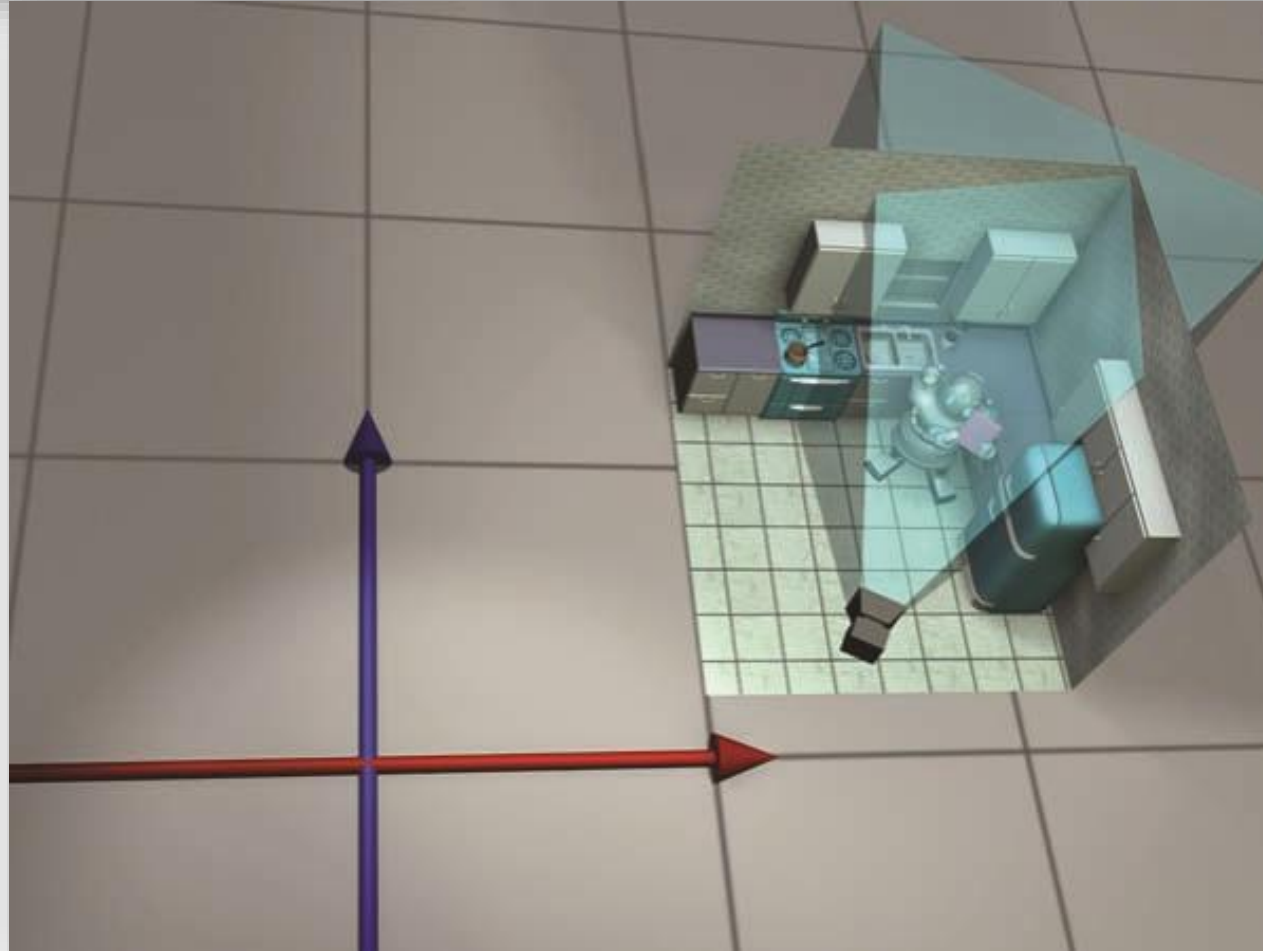


What the camera sees

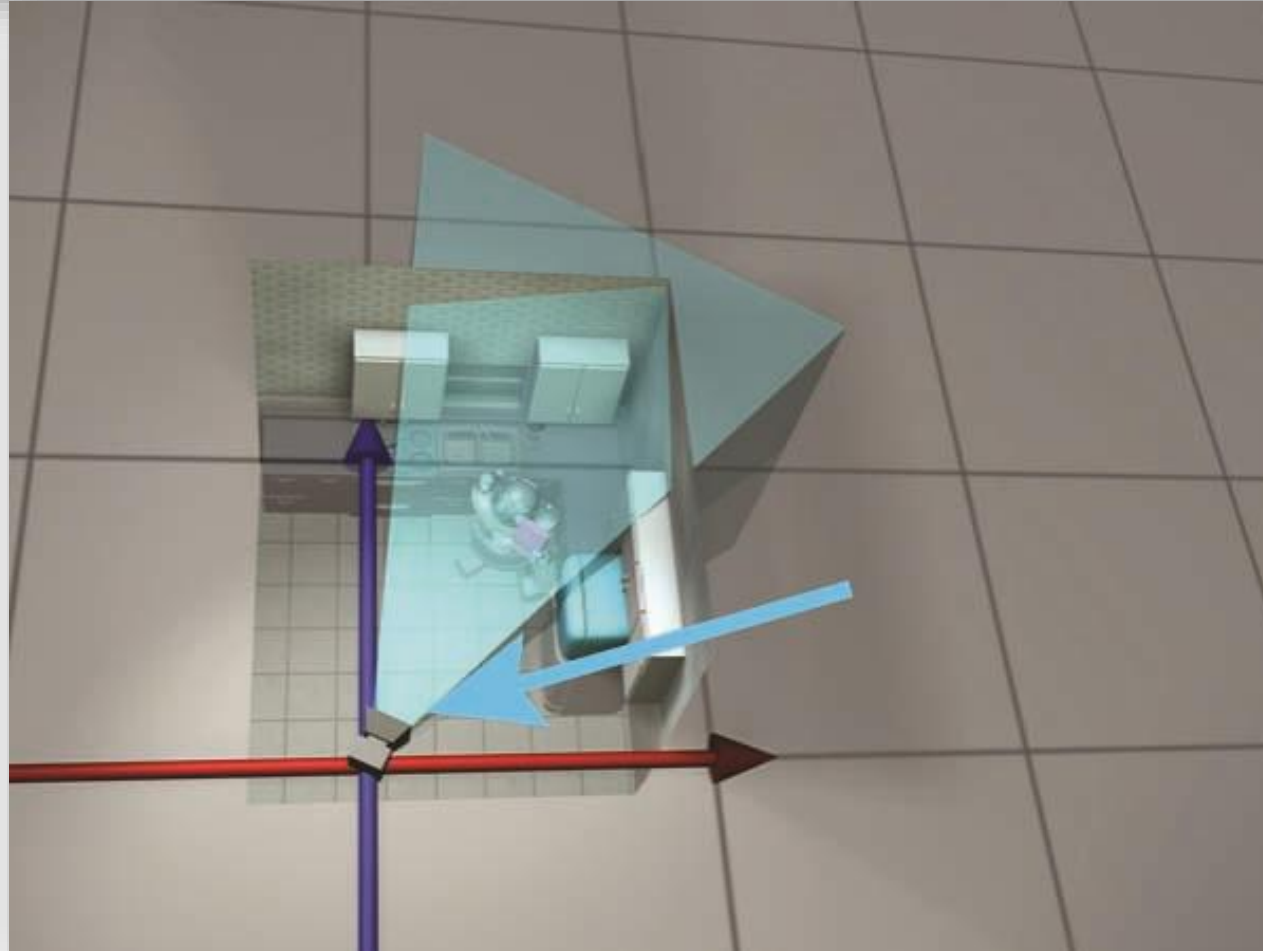
Camera Space

- It's easier to reason about a camera at the origin, looking along a primary axis.
- So we move the whole world so that the camera is at the origin.
- First we translate, then we rotate (for the same reason as before, because rotation about the origin is easier than rotation about an arbitrary point).

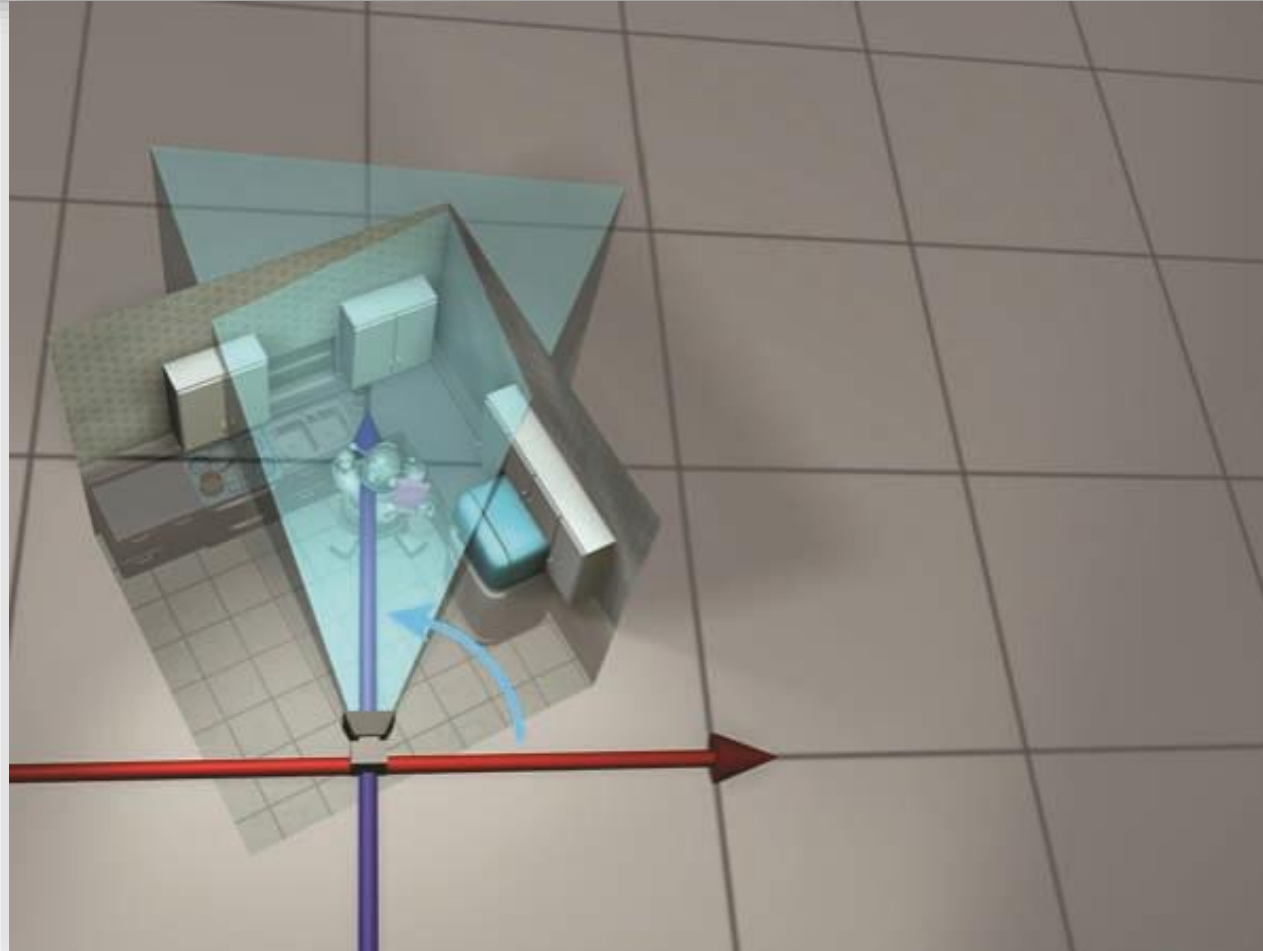
Original Position



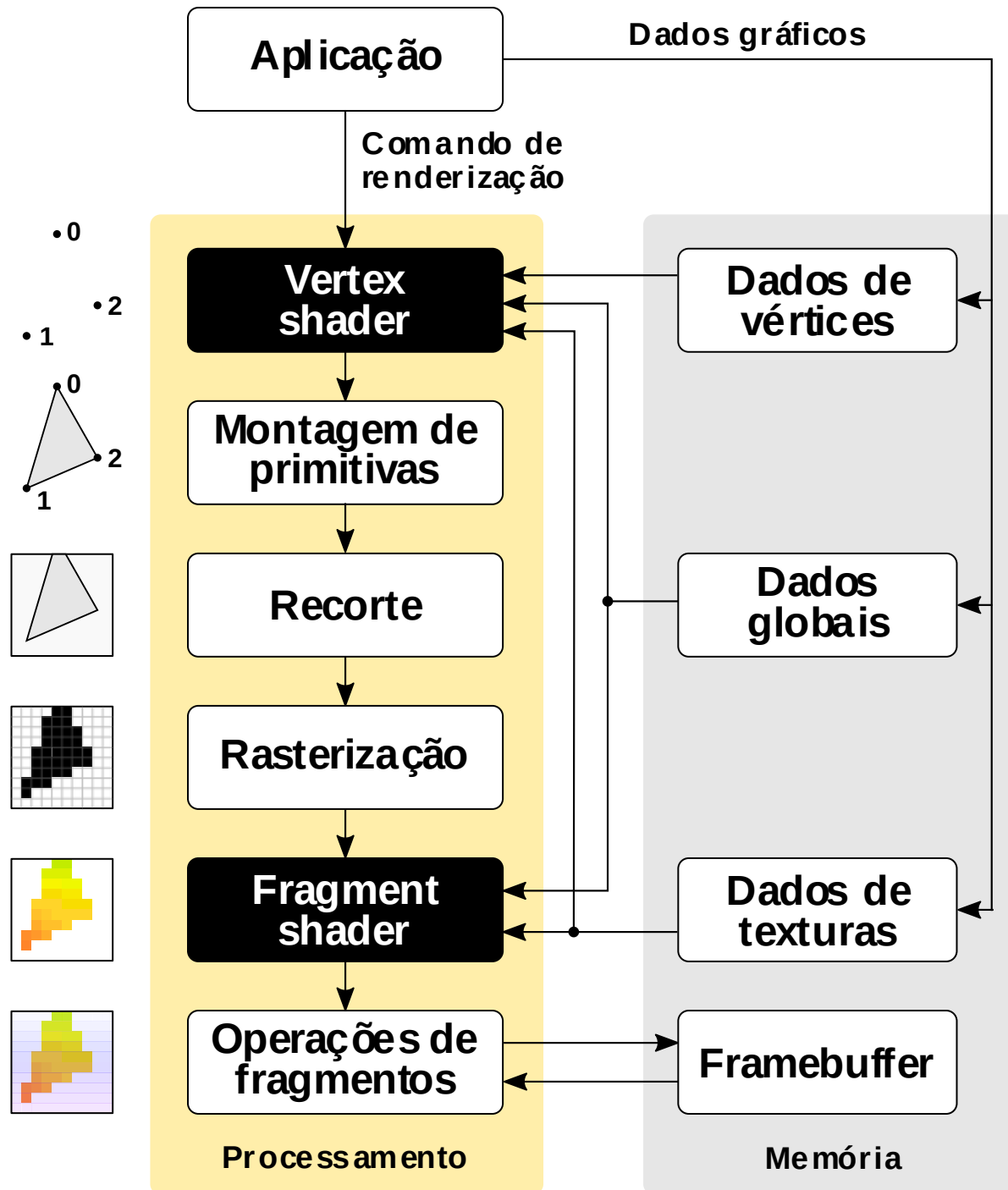
Translate the World



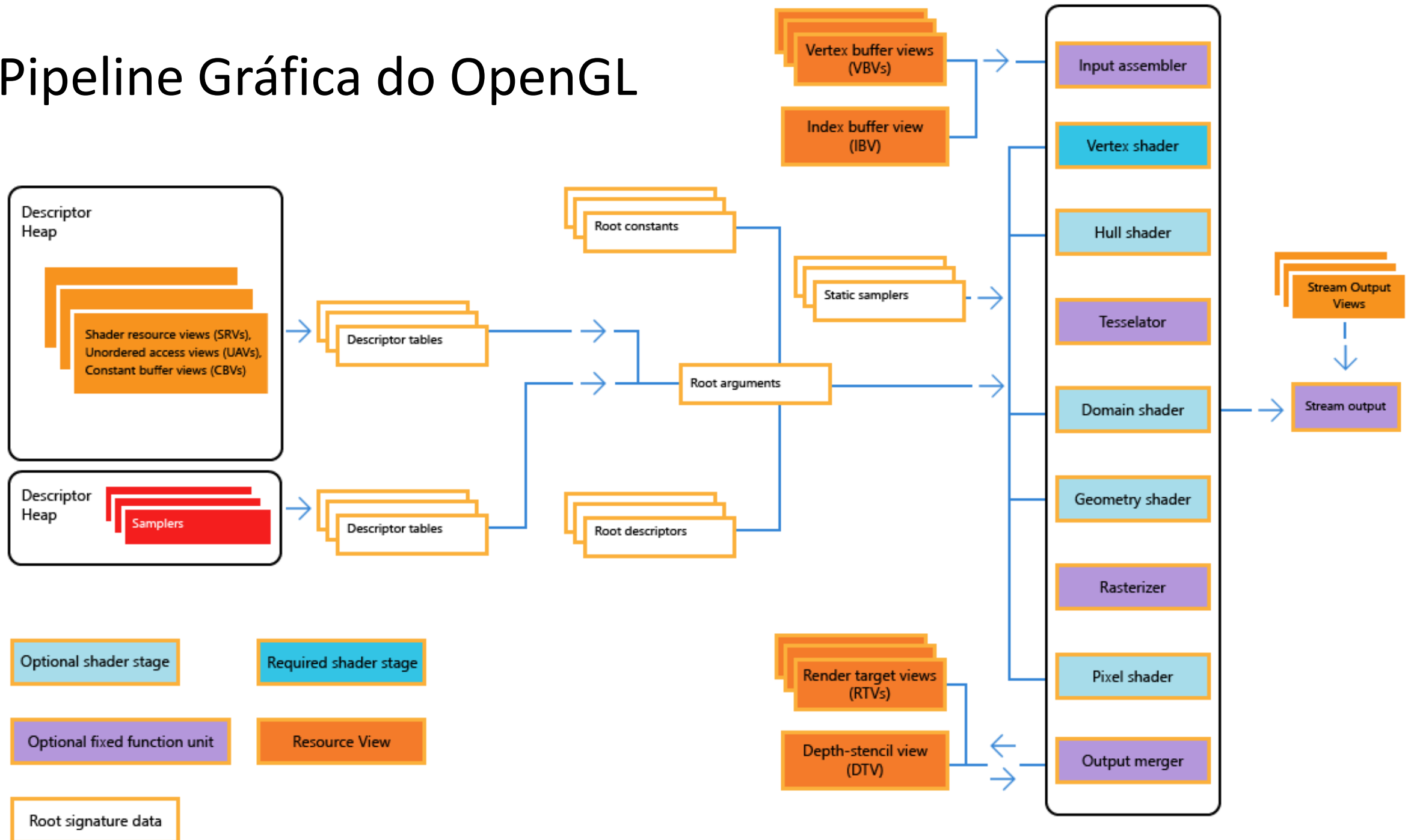
Rotate the World

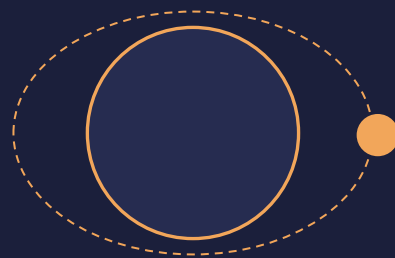


Pipeline Gráfica do OpenGL



Pipeline Gráfica do OpenGL





Obrigado!

Dúvidas?